WHO WROTE IT? IDENTIFYING LLMS FROM THEIR RESPONSES

CMPSC 448 - Machine Learning and AI
Midterm Project Report

Name: Ananya Reddy N (aqn5529)

Abstract

This project investigates whether text classifiers can identify which of six large language models (LLMs) generated a response. Using a balanced 12,000-example subset of the TXD-22 dataset, the study compares convolutional neural networks (CNNs) and bidirectional long short-term memory (LSTM) networks under three input configurations: prompt only, response only, and prompt plus response. The combined-input CNN achieved the highest accuracy (66.08%), while the response-only CNN achieved the highest macro-F1 (0.6526); these results were nearly tied. Prompt-only models performed close to the six-class chance baseline of 16.67%. Two additional analyses explored cross-domain generalization and text characteristics. A TF-IDF/Logistic Regression classifier achieved 87.25% accuracy on an in-domain non-computer-science test set and 81.55% on a held-out computer-science set. A classifier using six handcrafted text features achieved 55.18% accuracy and 0.5422 macro-F1. The findings suggest that generated responses contain detectable model-associated patterns in this dataset, but performance depends on the input representation and model architecture. Dataset scope, small held-out domain size, and differences between evaluation splits limit broader conclusions.

Keywords: large language models, text classification, CNN, LSTM, domain generalization, linguistic features

1. Introduction

Large language models (LLMs) can generate fluent answers across many topics, but models may differ in their vocabulary, phrasing, response length, sentence patterns, and formatting. These differences raise a practical question: can a machine-learning classifier use a response's characteristics to estimate which LLM produced it? This project evaluates that question using a public dataset and two neural architectures.

The report addresses four research questions. RQ1 asks whether the generated response can identify its source LLM. RQ2 tests whether the original prompt helps identification by comparing prompt-only, response-only, and combined inputs. RQ3 explores whether a classifier trained on one domain generalizes

to another. RQ4 examines measurable linguistic and structural characteristics that differ across model labels.

2. Dataset and Preprocessing

The project uses TXD-22, a publicly available benchmark dataset for multi-class AI-generated and mixed-source text identification. The dataset was obtained from Mendeley Data. A balanced subset of 12,000 examples was selected, with 2,000 examples for each of six labels: ChatGPT, Claude, DeepSeek, Gemini, Llama, and Qwen. Each example contains an input/prompt, a generated output/response, and a source-model label.

The neural experiments compared three text representations: the prompt alone, the response alone, and the prompt concatenated with the response. The data split grouped examples by normalized question to reduce the risk that repeated or closely matching prompts would appear in both training and test data. The main neural-model partitions contained 9,590 training examples, 1,210 validation examples, and 1,200 test examples. Since the six classes are balanced, a uniform random classifier would be expected to achieve 16.67% accuracy.

The balanced subset supports comparison across the six selected labels, but it is still a subset of one benchmark. The results therefore describe performance on this experimental dataset, not a universal ability to identify all LLMs or all future versions of these systems.

3. Methodology

3.1 CNN and LSTM architectures

Both required neural architectures were implemented in PyTorch. The CNN applies several convolutional filters over token sequences to learn local patterns such as short phrases and word combinations. The

LSTM processes token sequences recurrently; the implementation used a bidirectional LSTM so that representations can incorporate context from both directions.

For both architectures, the vocabulary was capped at 12,000 tokens, maximum sequence length was 160 tokens, and embedding dimension was 64. The CNN used filter kernel sizes 3, 4, and 5, with 64 channels per kernel. The LSTM used a bidirectional recurrent layer with hidden size 64. Both models used dropout of 0.3, the Adam optimizer with learning rate 0.001, batch size 128, and five epochs. The best checkpoint was selected according to validation macro-F1. Training used PyTorch's MPS backend on a Mac.

3.2 Evaluation metrics

Accuracy is the proportion of test examples classified correctly. Macro-F1 computes the F1 score separately for each class and then averages those six scores equally. Macro-F1 is useful because it does not let performance on one label dominate the metric. Both are reported for the neural models.

3.3 RQ3: Cross-domain generalization

For RQ3, a separate TF-IDF representation and Logistic Regression classifier was trained on non-computer-science examples. It was evaluated on an in-domain non-CS test set and a held-out computer-science subset. Examples were grouped using normalized-question identifiers. Domain labels were assigned using keyword-based rules, so this experiment should be interpreted as an exploratory transfer test rather than a definitive benchmark of domain generalization.

3.4 RQ4: Text-feature analysis

Six handcrafted features were extracted from each response: word count, lexical diversity, mean word length, sentence count, whether a bullet marker was present, and whether a triple-backtick code-block marker was present. Kruskal–Wallis tests compared the distributions of four numeric features across the six model labels. A separate Logistic Regression classifier used the six features, with missing-value imputation and standardization, and a grouped train/test split based on normalized questions.

4. Results: RQ1 and RQ2

Table 1 compares the CNN and LSTM across all three input settings. The combined-input CNN achieved the highest accuracy at 66.08%. The response-only CNN achieved the highest macro-F1 at 0.6526, narrowly above 0.6525 for the combined-input CNN. Because the two CNN results are almost identical, the data do not show a meaningful macro-F1 improvement from adding the prompt to the response.

Input configuration	Model	Accuracy	Macro-F1	Best validation Macro-F1
Prompt only	CNN	17.50%	0.1610	0.1539
Prompt only	LSTM	17.00%	0.1446	0.1500
Response only	CNN	65.92%	0.6526	0.6517
Response only	LSTM	54.50%	0.5304	0.5289
Prompt + response	CNN	66.08%	0.6525	0.6287
Prompt + response	LSTM	43.83%	0.3895	0.3666

Table 1. Test performance of the CNN and LSTM after five epochs. Best validation macro-F1 is included to show the validation checkpoint criterion; test metrics are the primary comparison.

Prompt-only accuracy was 17.50% for the CNN and 17.00% for the LSTM, both close to the 16.67% chance baseline. This suggests that the prompt content alone was not a reliable indicator of the model label in this subset. Response text was much more informative. The CNN outperformed the LSTM in all three settings. The combined-input LSTM performed worse than the response-only LSTM, showing that adding more text does not necessarily help every architecture.

The output-only CNN's 65.92% accuracy and 0.6526 macro-F1 indicate useful but imperfect classification. The result should not be interpreted as near-certain attribution: roughly one-third of test examples were still misclassified, and the experiment covers only six labels from the selected dataset.

5. Results: RQ3 Domain Generalization

Table 2 compares performance on the in-domain non-CS test set and the held-out computer-science subset. Accuracy decreased from 87.25% to 81.55%, a drop of approximately 5.70 percentage points. Macro-F1 decreased from 0.8709 to 0.8166, a difference of about 0.0543.

Evaluation set	Examples	Unique prompts	Accuracy	Macro-F1
In-domain non-CS test	2,360	604	87.25%	0.8709
Held-out CS set	168	41	81.55%	0.8166

Table 2. TF-IDF plus Logistic Regression performance when evaluated in-domain and on held-out computer-science examples.

The decrease suggests that some identifying patterns transfer to the held-out domain, but the transfer is not perfect. The held-out CS set is small, with 168 examples from 41 unique questions, so its estimate may vary substantially with the examples selected. In addition, keyword-based domain labeling can misclassify examples, and the results should not be generalized to every type of task.

6. Results: RQ4 Text Characteristics

Table 3 summarizes selected response characteristics. Average response length ranged from about 209 words for Llama to 297 words for Claude. Average lexical diversity ranged from 0.532 for Llama to 0.703 for Qwen. Mean word length and sentence count also varied across labels. These differences are descriptive signals, not proof that any single feature determines a model's identity.

LLM label	Mean words	Lexical diversity	Mean word length	Mean sentences
ChatGPT	246.6	0.626	5.875	10.772
Claude	297.3	0.617	6.409	7.833
DeepSeek	270.1	0.677	6.038	13.681
Gemini	268.2	0.616	5.784	11.234
Llama	209.5	0.532	5.558	7.855
Qwen	220.8	0.703	6.276	10.244

Table 3. Descriptive statistics for selected response features by LLM label. Lexical diversity is the ratio of unique word tokens to total word tokens as computed by the project feature extractor.

The Kruskal–Wallis tests found statistically significant differences across the six labels for each of the four numeric features tested, with all p-values below .001 (Table 4). The CSV output displayed these p-values as 0.0 because of numerical underflow; a probability of exactly zero should not be reported. These omnibus tests indicate that at least some label distributions differ, but they do not identify which pairs differ, quantify practical importance, or establish causation.

Feature	Kruskal–Wallis H	p-value
Word count	2976.19	< .001
Lexical diversity	4555.03	< .001
Mean word length	2961.35	< .001
Sentence count	3209.59	< .001

Table 4. Kruskal–Wallis tests comparing numeric feature distributions across the six model labels. All reported p-values should be interpreted as $p < .001$.

The handcrafted-feature Logistic Regression classifier achieved 55.18% accuracy and 0.5422 macro-F1 (Table 5). This is above the balanced six-class chance baseline, indicating that simple length and lexical features contain some identifying information. However, it is below the strongest CNN result. This comparison is suggestive rather than controlled because the handcrafted-feature classifier and neural CNN were evaluated on different grouped test splits.

Classifier	Train n	Test n	Test prompts	Accuracy	Macro-F1
Logistic Regression using six handcrafted features	9,604	2,396	612	55.18%	0.5422

Table 5. Overall performance of Logistic Regression using the six handcrafted response features.

The per-class scores in Table 6 show uneven performance. The classifier had higher F1 scores for Claude (0.812) and Llama (0.790), but lower scores for ChatGPT (0.299) and Gemini (0.322). Thus, the same features were more useful for some labels than others.

LLM label	Precision	Recall	F1-score	Support
ChatGPT	0.358	0.257	0.299	397
Claude	0.786	0.841	0.812	402
DeepSeek	0.562	0.582	0.572	397
Gemini	0.339	0.307	0.322	410
Llama	0.747	0.840	0.790	393
Qwen	0.428	0.491	0.457	397
Macro average	0.536	0.553	0.542	2,396

Table 6. Per-class performance of the handcrafted-feature classifier. Support is the number of test examples for each label.

The bullet-marker and code-block indicators were zero for all six labels in the summarized feature output. Therefore, these indicators did not distinguish the classes in this dataset subset. No controlled feature-removal ablation was conducted, so the analysis cannot establish how much each feature contributed to the classifier or what internal signals the CNN learned.

7. Discussion

Across the main experiments, generated response text was more informative than the prompt alone. The prompt-only models remained close to chance, whereas response-based CNNs achieved approximately 66% accuracy. The combined-input CNN had the highest accuracy, but its macro-F1 was effectively tied with the response-only CNN. The LSTM results also show that adding the prompt can hurt performance, depending on the architecture.

The additional experiments provide a broader view of model-associated patterns. The TF-IDF classifier's performance decreased on held-out computer-science examples but remained above 80% accuracy. Handcrafted features provided moderate predictive power and showed statistically significant distributional differences, although they did not match the strongest neural model. These findings are consistent with detectable stylistic and lexical differences in the dataset, but they do not prove that the classifier relies on any one particular signal.

8. Lessons Learned

This project provided experience in preparing a balanced text dataset, implementing CNN and LSTM models, and evaluating classification performance with both accuracy and macro-F1. Comparing input configurations showed that the generated response contained much more useful information for identifying the source model than the prompt alone. It also demonstrated that adding more input does not automatically improve performance.

The extra-credit experiments highlighted the importance of evaluating models beyond a single test setting and checking what simpler features can explain. Statistical significance alone does not mean that a feature-based classifier will be highly accurate, and domain-transfer results depend on the size and quality of the held-out sample. The project reinforced the value of grouped data splitting, reproducible experiment settings, multiple metrics, and careful interpretation of limitations.

9. Limitations

This study uses one public dataset subset and six model labels. Results may not generalize to newer versions, different prompts, longer multi-turn conversations, or text from other domains. Grouping by normalized question reduces leakage from repeated prompts but cannot eliminate all dataset bias or overlap.

The neural models were trained for five epochs with fixed hyperparameters; additional tuning might change performance. The RQ3 held-out set is small and uses heuristic domain labels. The RQ4 feature definitions are simple, and the statistical tests do not establish causal relationships. The handcrafted-feature classifier and CNN use different test splits, so their metrics are not a strictly controlled comparison. The current feature analysis also does not include an ablation study or pairwise post-hoc tests.

10. Conclusion

This project evaluated whether CNN and LSTM classifiers could identify the source of text generated by six LLMs. The strongest results came from response text: the combined-input CNN achieved 66.08% accuracy, and the response-only CNN achieved a macro-F1 of 0.6526. Prompt-only models remained near chance. Additional analyses found a modest decrease in performance when transferring a TF-IDF classifier to held-out computer-science examples and showed that handcrafted text features contain some

predictive signal. Overall, the experiments support the existence of model-associated patterns in this dataset, while also showing that identification is imperfect and sensitive to experimental conditions.

References

Iqbal, M. S., & Kashem, M. A. (2026). TXD-22: A Large-Scale Benchmark Dataset for Multi-Class AI-Generated and Mixed-Source Text Identification (Version 1). Mendeley Data. https://doi.org/10.17632/prcjcggtjf.1

PyTorch. (n.d.). PyTorch documentation. https://pytorch.org/docs/

scikit-learn developers. (n.d.). scikit-learn documentation. https://scikit-learn.org/stable/

Appendix A. Reproducibility

The GitHub repository contains the data preparation and training scripts, additional analysis scripts, results CSV files, and README instructions. The principal neural experiment used a vocabulary cap of 12,000, sequence length 160, embedding dimension 64, batch size 128, five epochs, Adam learning rate 0.001, and dropout 0.3. The RQ3 and RQ4 experiments were run separately from the CNN/LSTM comparison. The reported values correspond to the recorded output files for these experiments.